

ELO329 - Diseño y Programación Orientados a Objetos

Ingeniería de Software

Agustín J. González
Patricio Olivares R.

Universidad Técnica Federico Santa María

Propósito de la clase

Hasta ahora hemos hablado bastante de clases, objetos, herencia, eventos e interfaces.

El siguiente problema aparece cuando el software deja de ser una tarea individual y pasa a ser un producto que debe construirse, probarse, entregar y mantener.

La pregunta cambia: "¿cómo organizo el desarrollo para que el producto llegue a tiempo, funcione y pueda cambiar?".

Ruta

En esta clase conectaremos cuatro ideas:

1. Qué entendemos por Ingeniería de Software.
2. Cómo el SDLC ordena el ciclo de desarrollo.
3. Cómo elegir entre metodologías tradicionales, iterativas y ágiles.
4. Por qué las licencias son parte de las decisiones de ingeniería.

El hilo conductor será pasar de código escrito por una persona a software construido por un equipo para usuarios reales.

De programa a producto

No es lo mismo resolver una tarea pequeña que construir un sistema que otros usarán.

Tarea pequeña	Producto de software
alcance acotado	alcance cambiante
una persona puede entender todo	varios roles deben coordinarse
poco costo de mantenimiento	errores y cambios tienen impacto real
entrega única	versiones, soporte y evolución

Mientras más grande y cambiante es el producto, más importante se vuelve el proceso.

Ingeniería de Software

La Ingeniería de Software aplica un enfoque sistemático, disciplinado y medible al desarrollo de software.

Busca controlar tres tensiones permanentes:

- resolver una necesidad real.
- hacerlo con tiempo y costo razonables.
- sostener calidad, seguridad y operación.

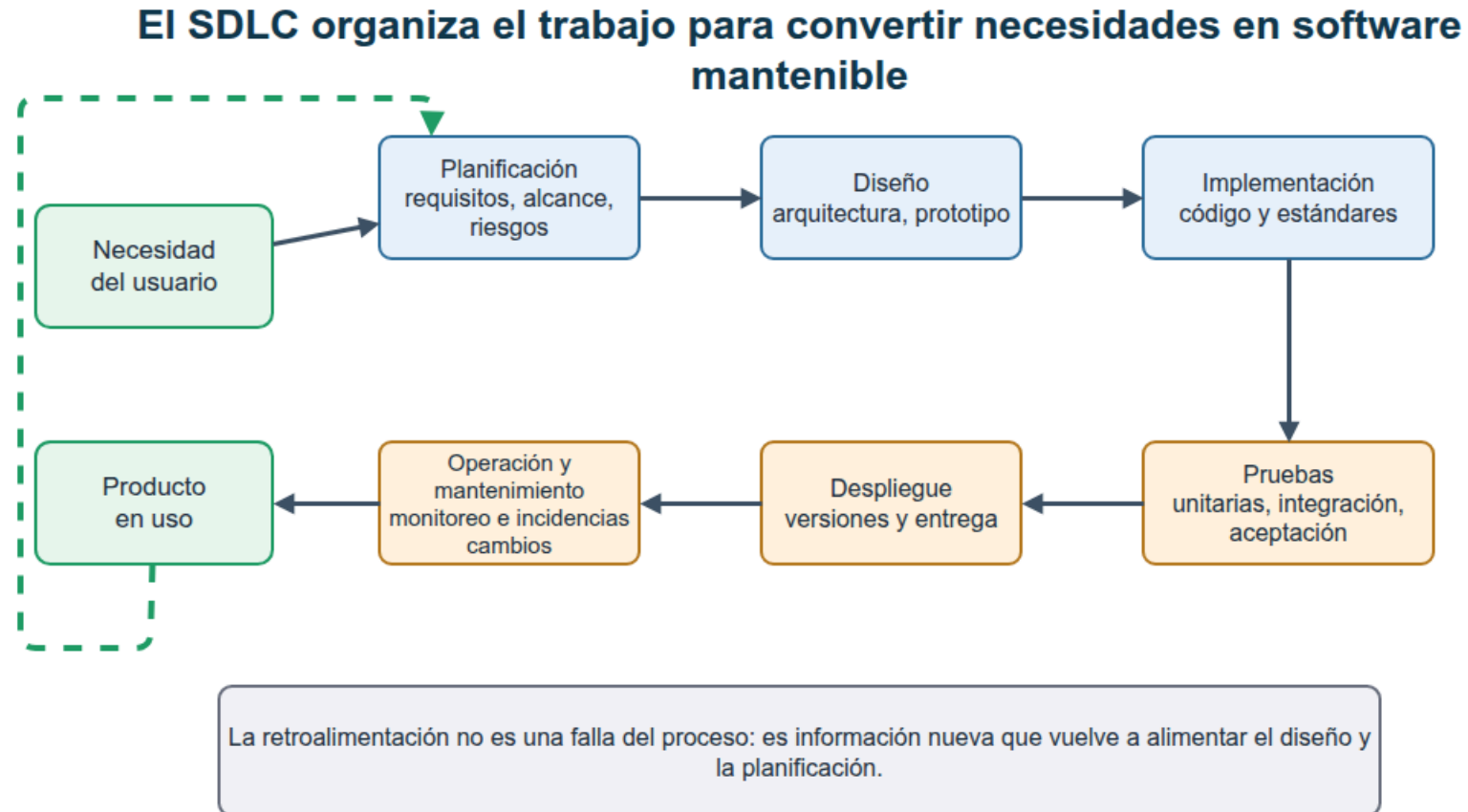
El software incluye código, requisitos, arquitectura, pruebas, despliegue, operación y mantenimiento.

Objetivos de calidad

Una metodología no reemplaza el criterio técnico. Ayuda a sostenerlo.

Objetivo	Pregunta práctica
Calidad	¿cumple los criterios de aceptación?
Reusabilidad	¿puedo aprovechar módulos en otro contexto?
Testeabilidad	¿puedo verificar componentes por separado?
Adaptabilidad	¿puede evolucionar cuando cambien requisitos?
Portabilidad	¿puede moverse a otro entorno razonablemente?

Ciclo de vida de desarrollo



SDLC significa *Software Development Life Cycle*: ciclo de vida de desarrollo de software.

Define quién hace qué, cuándo y con qué entregables.

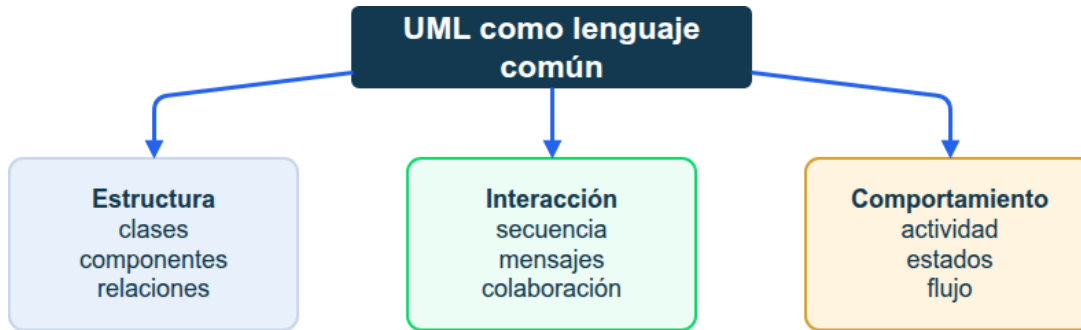
SDLC en seis preguntas

Etapa	Pregunta
Planificación	¿qué problema se resolverá y bajo qué restricciones?
Diseño	¿qué arquitectura y componentes sostendrán la solución?
Implementación	¿cómo se traduce el diseño a código mantenible?
Pruebas	¿cómo sabemos que cumple lo esperado?
Despliegue	¿cómo llega una versión a usuarios reales?
Operación y mantenimiento	¿cómo se observa, corrige, mejora y evoluciona?

Términos en el flujo del proyecto

Momento	Término	Para qué aparece
Planificación y requisitos	SRS	documentar qué debe cumplir el sistema
Diseño y análisis	UML	discutir estructura e interacción del sistema
Pruebas y validación	QA, UAT	asegurar calidad y validar con usuarios
Entrega temprana	MVP	probar valor con una versión mínima
Despliegue y operación	CI/CD, DevOps	automatizar entrega y mantener el sistema funcionando

Diagramas UML



UML significa *Unified Modeling Language*: lenguaje de modelado unificado.

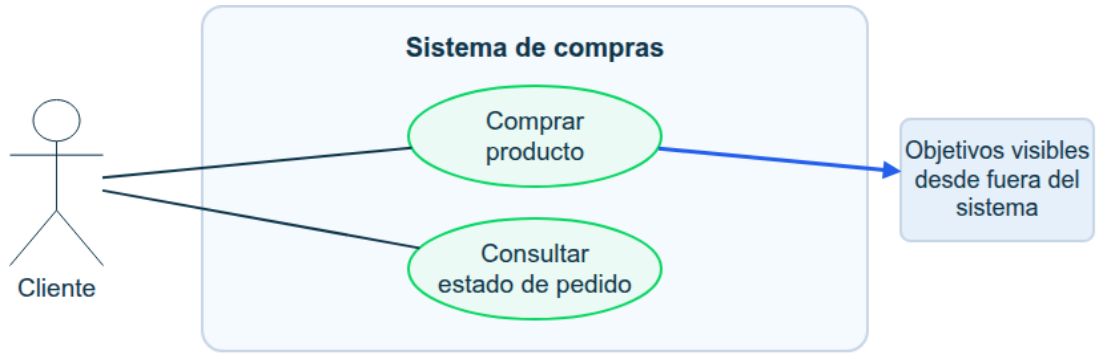
No es una metodología. Es una notación para comunicar vistas del sistema.

Ayuda a representar:

- estructura del software.
- interacción entre componentes.
- comportamiento esperado.

En el hilo de la clase, UML ayuda a pasar de una idea conversada a un diseño que el equipo puede discutir.

Casos de uso



Un caso de uso describe una meta visible de un actor frente al sistema.

Elementos mínimos:

- actor: rol externo al sistema.
- sistema: límite de lo que se construye.
- caso de uso: objetivo que entrega valor.

Sirve para conversar alcance con el cliente.

Historias de usuario



Una historia debe ser pequeña, verificable y priorizable

Una historia de usuario resume una necesidad pequeña y priorizable.

Formato habitual:

Como <rol> quiero <acción> para <beneficio>

Comparación breve:

- caso de uso: interacción más completa.
- historia: unidad pequeña para backlog.
- ambas conectan cliente, equipo y producto.

Metodología de desarrollo

Una metodología define procedimientos, roles y artefactos para ordenar el trabajo.

Ejemplos:

- cascada y modelo V.
- espiral.
- iterativo e incremental.
- metodologías ágiles como Scrum o Kanban, que usa tableros para visualizar flujo de trabajo.

La diferencia está en cuánto planifican por adelantado, cómo participa el cliente y cuán rápido incorporan retroalimentación.

Metodologías tradicionales

En cascada, cada fase entrega la entrada para la siguiente.

En modelo V, las fases de definición se conectan explícitamente con fases de prueba y validación.

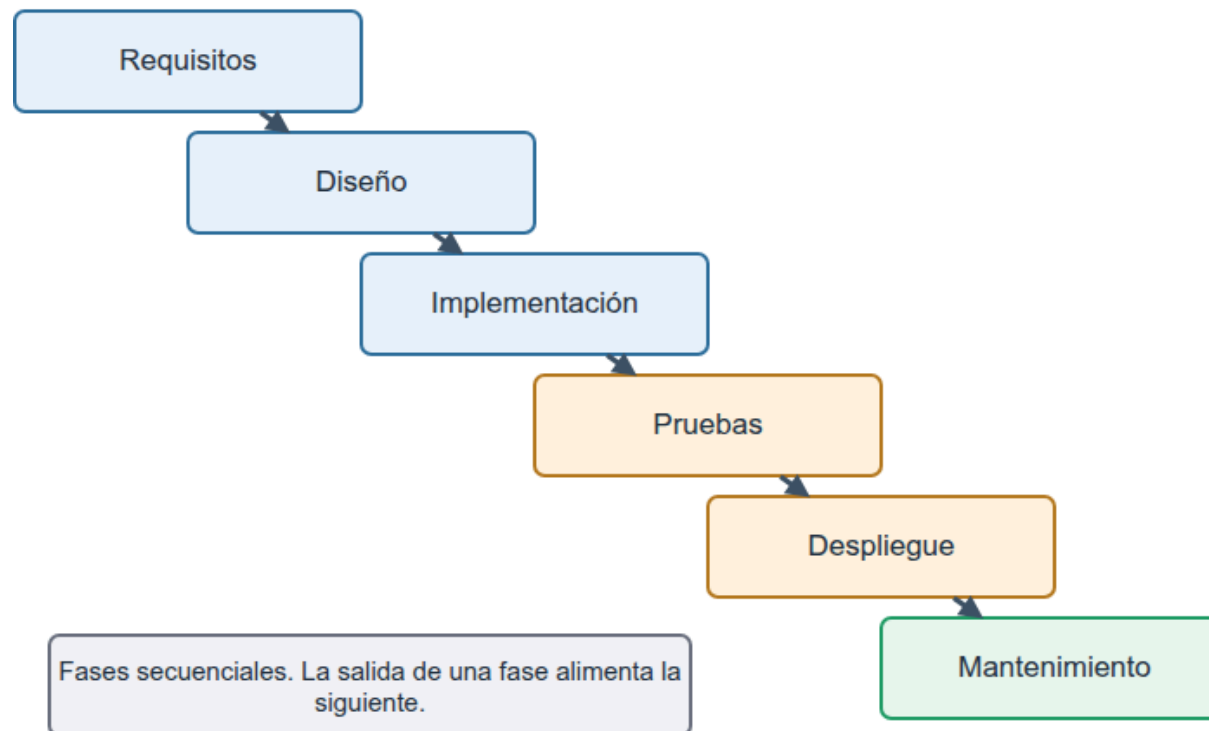
La participación del cliente suele concentrarse en requisitos iniciales, revisiones formales y aceptación.

Funcionan mejor cuando:

- los requisitos son estables.
- hay alta necesidad de trazabilidad.
- el costo de cambiar tarde es muy alto.
- el entorno exige documentación formal.

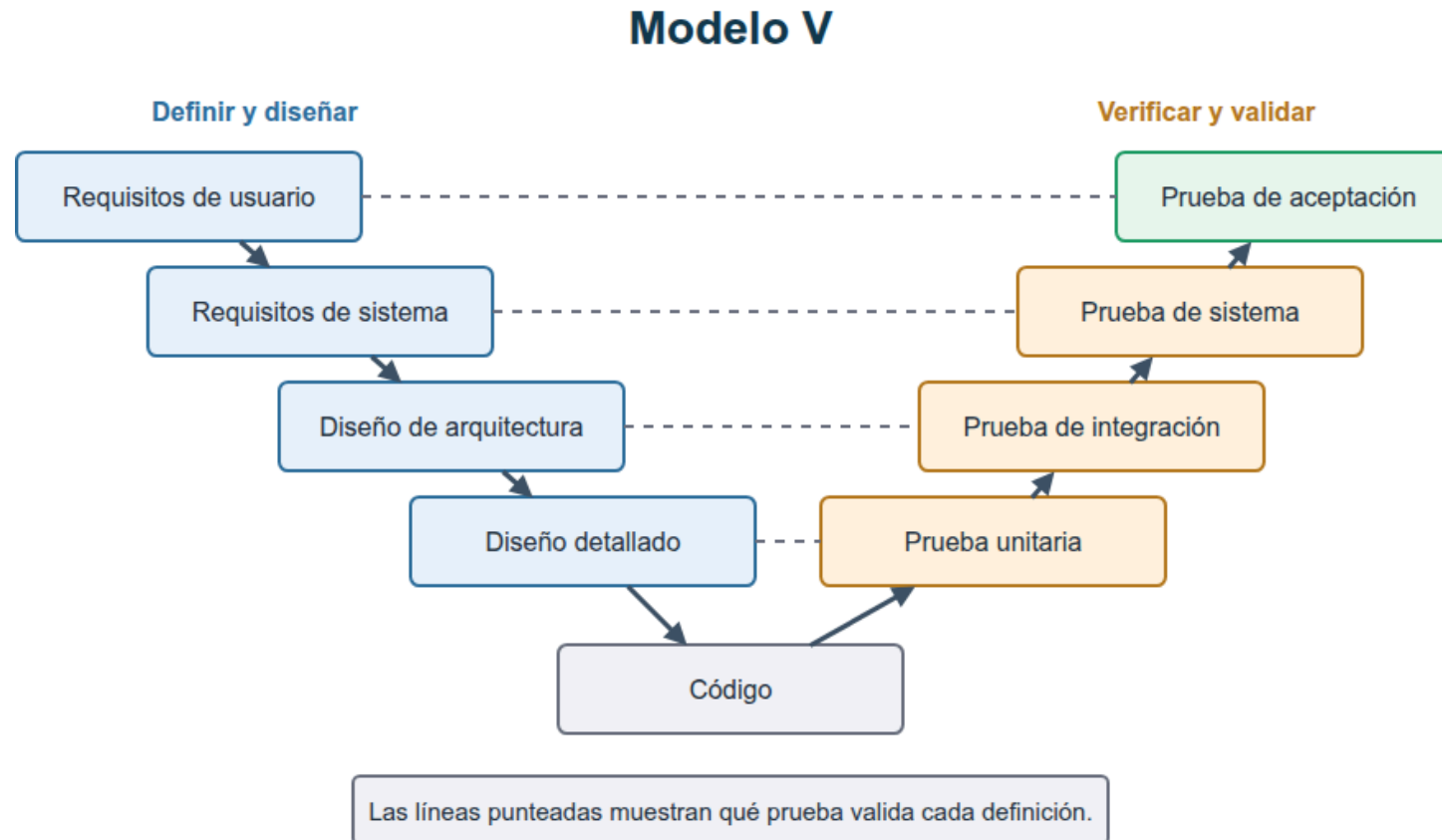
Modelo en cascada

Modelo en cascada



Útil cuando los requisitos son claros y se necesita trazabilidad.

Modelo V



Cada definición se vincula con una prueba posterior.

Iterativo e incremental

El desarrollo iterativo vuelve varias veces sobre análisis, diseño, implementación y pruebas.

El desarrollo incremental entrega versiones sucesivas con valor parcial.

Iterar permite aprender. Incrementar permite entregar.

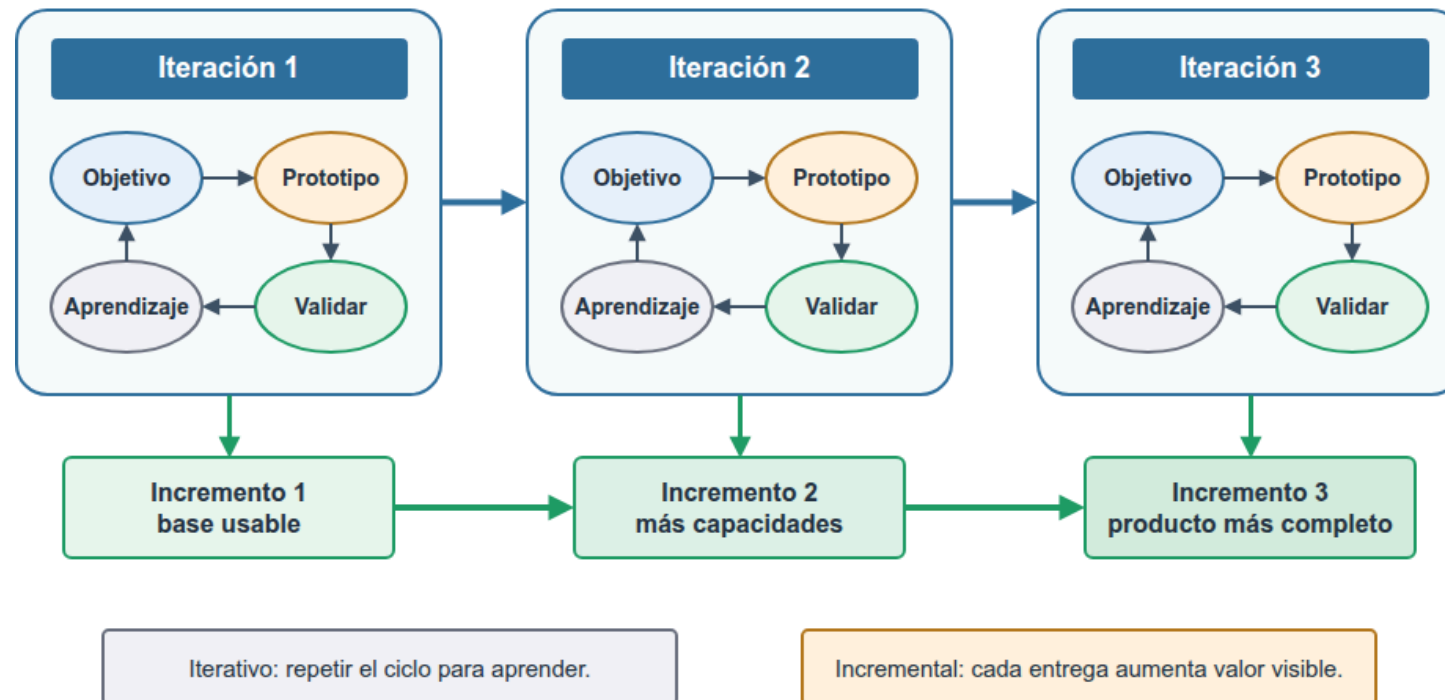
Cada iteración debería dejar un prototipo o incremento que el cliente pueda evaluar.

Ejemplo: una versión mínima funcional, luego nuevas funcionalidades, luego mejoras de integración.

Iteraciones e incrementos

Desarrollo iterativo incremental

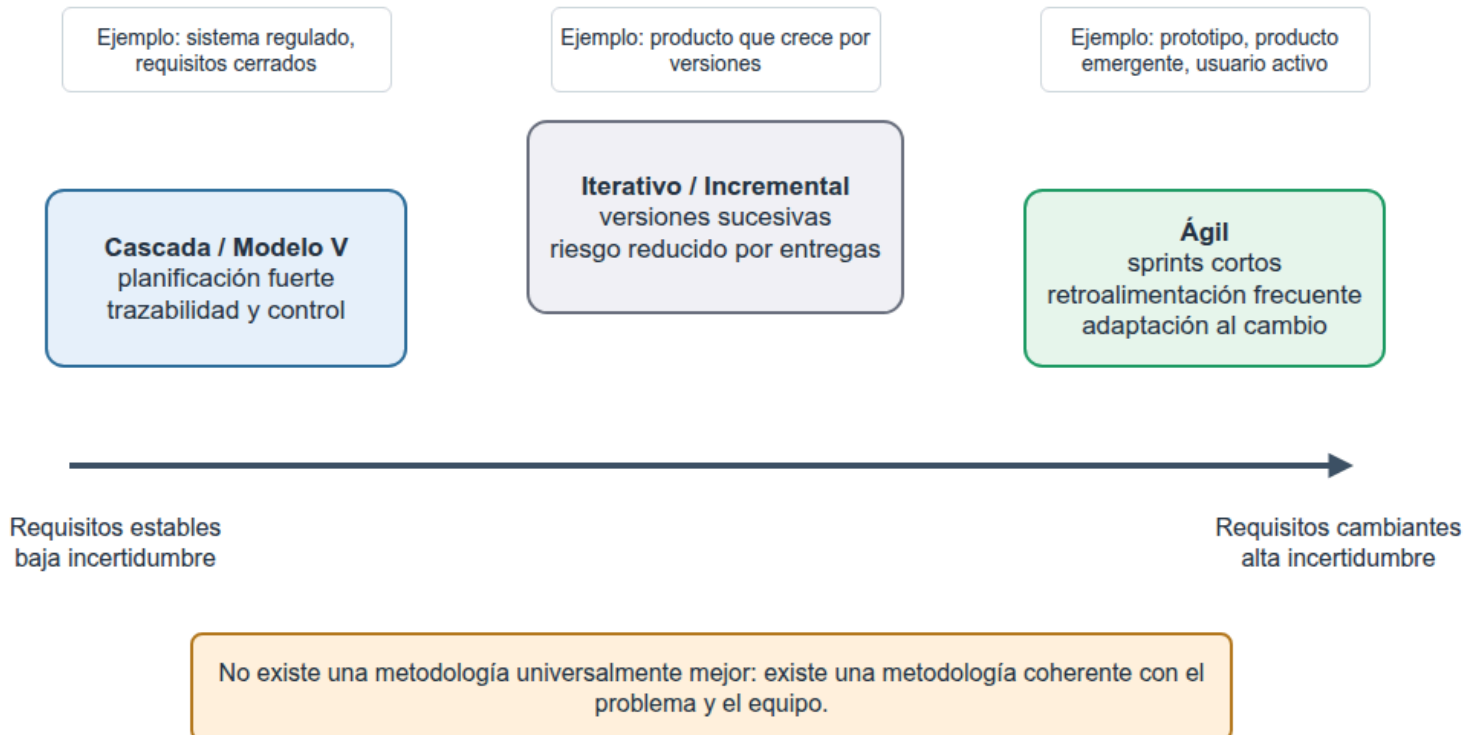
Cada ciclo aprende del anterior y entrega un producto más útil



Cada ciclo entrega algo usable, recibe evaluación del cliente y deja aprendizaje para el siguiente.

Elegir metodología

La metodología se elige según incertidumbre, riesgo y necesidad de retroalimentación



La elección depende del producto, del equipo, del riesgo y de la incertidumbre.

Metodologías ágiles

Ágil es un enfoque iterativo: ciclos cortos, software funcionando y aprendizaje frecuente.

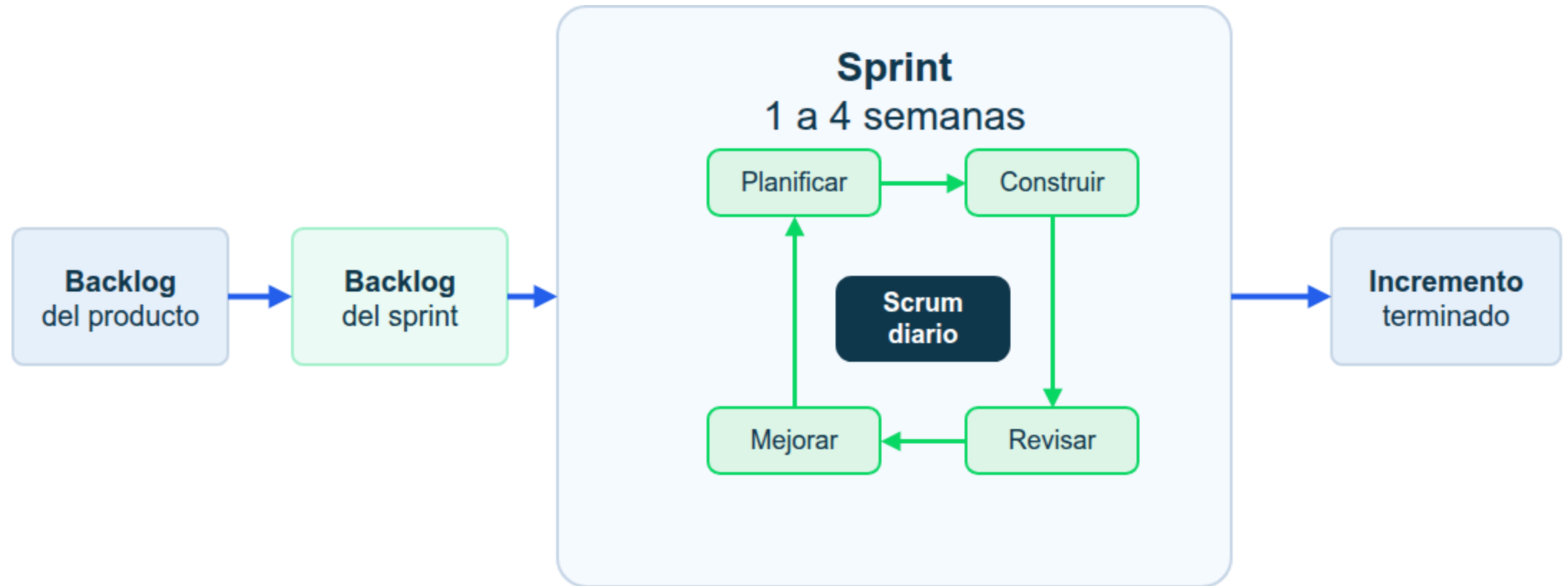
Se busca:

- requisitos pueden evolucionar.
- retroalimentación temprana reduce riesgo.
- pruebas y revisión ocurren en cada ciclo.
- usuarios o clientes revisan incrementos y ajustan prioridades.

Ágil no significa improvisar. Significa planificar en ciclos cortos y ajustar con evidencia.

Scrum se basa en empirismo y pensamiento lean: aprender desde evidencia y reducir desperdicio.

Ciclo ágil



Cada sprint debe terminar con algo demostrable o con aprendizaje útil para decidir el siguiente paso.

Scrum mínimo

Scrum organiza el trabajo con un backlog priorizado y sprints cortos.

Sprint: ciclo corto de trabajo. *Product Backlog*: lista priorizada del producto.
Los nombres de roles se mantienen como aparecen en la guía oficial de Scrum.

Elemento	Rol
Product Owner	"dueño del producto", maximiza valor y ordena el Product Backlog
Scrum Master	establece Scrum y ayuda a remover impedimentos
Developers	desarrolladores, crean un incremento usable en cada sprint
Sprint Review	muestra avance y recibe retroalimentación
Retrospectiva	mejora la forma de trabajar

Riesgos frecuentes

Ágil falla cuando se reduce a reuniones sin producto funcionando.

También falla cuando:

- no hay prioridades claras.
- el backlog no representa valor para usuarios.
- se posponen pruebas hasta el final.
- se confunde cambio con falta de dirección.

La disciplina ágil está en cerrar ciclos completos: decidir, construir, probar, mostrar y ajustar.

De construir a integrar

En un proyecto real no todo se escribe desde cero.

- se incorporan bibliotecas, frameworks, plantillas, servicios y ejemplos.
- cada dependencia trae ventajas técnicas y condiciones de uso.
- elegir una dependencia también es decidir riesgo, mantenimiento y cumplimiento.
- no basta con compilar y pasar pruebas si después no se puede distribuir.

Una metodología ordena cómo construimos. Las licencias ayudan a decidir qué podemos integrar y entregar.

Licencias de software

Una licencia define qué puede hacer un equipo con software de terceros.

Responde preguntas de ingeniería:

- uso: incorporarlo en el proyecto.
- modificación: cambiarlo o adaptarlo.
- distribución: entregar el producto a otros.
- publicación: decidir si debe abrirse parte del código.

Que el código esté disponible no significa automáticamente que pueda usarse libremente.

Open source no significa solo "código visible": también permite redistribuir, modificar y usar sin discriminar personas ni usos.

Lo que debe saber el equipo

No se espera que un estudiante se vuelva experto legal en licencias.

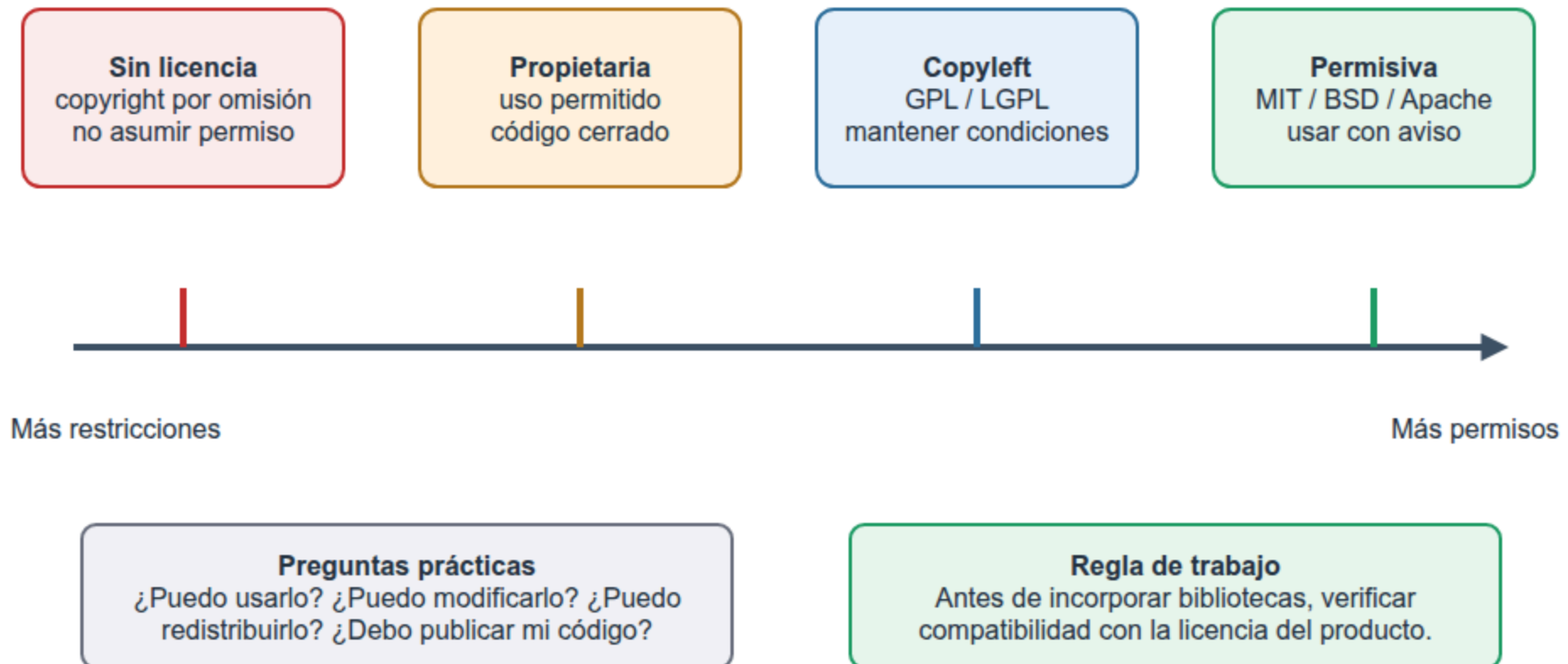
Sí se espera que pueda reconocer cuándo una decisión técnica puede afectar:

- diseño del producto.
- uso de bibliotecas o frameworks.
- distribución del software.
- obligaciones sobre el código fuente.

Ante consultas legales, uso comercial o distribución pública, deben acudir a un abogado antes de decidir.

Mapa de licencias

Una licencia define qué se puede hacer con el software y su código



Mapa de licencias

Antes de incorporar bibliotecas o frameworks, se debe revisar si sus licencias son compatibles con el producto que se quiere construir.

La pregunta de ingeniería no es solo si la biblioteca funciona, sino si puede usarse en el contexto del producto.

Licencias comunes

Tipo	Idea principal	Ejemplos
Sin licencia	no hay permisos explícitos y opera copyright por defecto	repositorios sin <code>LICENSE</code>
Propietaria	permite uso bajo condiciones, sin liberar código	software comercial cerrado
Copyleft fuerte	derivados deben mantener condiciones y liberar código	GPL
Copyleft débil	permite ciertos usos como biblioteca bajo condiciones	LGPL
Permisiva	permite usar, modificar y redistribuir con aviso	MIT, BSD, Apache
Muy permisiva	permite usar, modificar y redistribuir casi sin restricciones	WTFPL

GPL significa General Public License. LGPL es Lesser GPL. MIT, BSD y Apache son licencias permisivas comunes. WTFPL se suele citar como un ejemplo extremo de licencia permisiva. Su tono informal no la vuelve adecuada para todo proyecto.

Referencias actuales

- IEEE Computer Society, [SWEBOK Guide V4.0/V4.0a: Software Engineering Body of Knowledge](#).
- Scrum Guides, [The 2020 Scrum Guide](#).
- Open Source Initiative, [The Open Source Definition](#).
- [Choose a License](#).